

Program- It is a set of instructions to solve a particular problem with the help of logical and mathematical approach.

Algorithm- It is a step-by-step procedure to solve a problem. First, we have to gather the information according to the problem definition.

Characteristics of an Algorithm:

1. **Finiteness:** After a finite number of steps, an algorithm must always stop.
2. **Definiteness:** An algorithm has to include detailed descriptions for each step. For the instructions to be implemented without confusion
3. **Input:** The data required to carry out the stages in an algorithm can be zero or more inputs.
4. **Output:** The output should to be relevant to the issue being resolved and, ideally, needs to meaningfully convey the solution.
5. **Effectiveness:** This feature guarantees that the algorithm can be used with simple computing procedures without the need for sophisticated instruments.

THE PROBLEM-SOLVING ASPECT

1. Understanding the Problem

- Clearly define the problem's characteristics, boundaries, and extent.

2. Analysis

- Divide the problem into simpler to handle, smaller segments, or sub-problems.
- Choose the appropriate algorithms (such as sorting, searching, or optimization) and data structures

3. Design

- Create a high-level plan or architecture for the solution. This may include flowcharts, pseudo code, or UML diagrams.
- To test ideas and get input, create a basic version of the solution.

4. Implementation

- A programming language should be used to write the actual code.
- Implement system, integration, and unit tests to make sure the solution functions as intended.

5. Evaluation

- Analyse the solution's effectiveness in relation to the complexity of time and space.
- Verify that the solution satisfies the initial specifications and resolves the issue.

TOP-DOWN APPROACH

The top-down approach is a method of problem-solving that begins with a high-level summary of a system or issue and gradually dissects it into smaller, more specific parts.

Numerous disciplines, such as computer science, software engineering, and project management, make extensive use of this approach.

Below is a summary of its main features:

1. High-Level Overview

Start by comprehending the system or problem in its entirety. This entails determining the main aims and objectives without getting bogged down in the specifics.

2. Decomposition

Divide the issue into smaller portions or sub-problems. Every sub-problem need to reflect a distinctive aspect of the primary problems.

3. Hierarchical Structure

Tools such as hierarchical diagrams and flowcharts can be used to visualize this structure.

4. Design and Implementation

Establish solutions for each of the sub-problems after they have been identified. The next step is implementation, in which every component is created and incorporated into the system as a whole.

5. Testing and Validation

To make sure that every part functions as intended, test each component separately (unit testing) before testing the entire system.

Advantages of Top-Down Approach

- **Clarity:** Provides a clear understanding of the overall structure and goals, making it easier to manage complexity.
- **Focus:** Allows teams to concentrate on one aspect of the problem at a time, reducing cognitive overload.
- **Modularity:** Facilitates modular design, where components can be developed, tested, and modified independently.

IMPLEMENTATION OF ALGORITHMS

Steps for Implementing Algorithms

- 1. Understand the Algorithm:**
 - Read the algorithm i.e. learn about its function and operation.
 - Determine its inputs, outputs, and any potential side effects.
- 2. Choose a Programming Language:**
 - Select a language that suits your needs (e.g. Python, Java, C++, etc.).
- 3. Set Up Your Development Environment:**
 - Make sure you have the necessary tools, such as an IDE or text editor and a compiler/interpreter.
- 4. Write the Code:**
 - Start translating the algorithm into code.

- Use meaningful variable names and comments to clarify your logic.
- 5. **Test the Implementation:**
 - Run the code with various test cases, including edge cases, to ensure it works as expected.
 - Use assertions or unit tests to validate correctness.
- 6. **Optimize if Necessary:**
 - Analyse the performance and consider improvements if needed (e.g., time complexity, space complexity).
- 7. **Document the Code:**
 - Write clear documentation to explain how to use the algorithm and any assumptions made.

THE EFFICIENCY AND ANALYSIS OF ALGORITHM

The effectiveness of an algorithm in terms of time and space as the size of the input increases is known as algorithm efficiency. Determining an algorithm's feasibility for usage in practical applications requires analysing its efficiency, particularly as the size of the input rises.

Analysis focuses on two primary aspects of efficiency:

1. **Time Complexity:** how the algorithm's execution time grows as the size of the input rises.
2. **Space Complexity:** how the algorithm's memory consumption rises as the size of the input increases.

Time Complexity

Time complexity provides an estimate of how long an algorithm takes to execute based on the size of the input.

- **Big O Notation (O):** This notation, which specifies an upper bound on an algorithm's execution time, is the most widely used method of expressing temporal complexity. By providing the worst-case scenario, it aids in our comprehension of the algorithm's maximum execution time as the size of the input increases.

Ex:

- **O(1):** Constant time: No matter how big the input is (for example, accessing an element in an array), the algorithm takes the same amount of time.
- **O(n):** Linear time: When an algorithm iterates through an array, for example, its runtime grows linearly with the size of the input.
- **O(n²):** quadratic time, where the runtime increases in proportion to the input size squared (e.g., bubble sort).
- **O(log n):** Logarithmic time: For example, binary search, the runtime increases logarithmically with the size of the input.
- **O(n log n):** Log-linear time, or: is how the runtime increases (e.g., merging sort, quicksort).

Space Complexity

The term "space complexity" describes how much memory an algorithm requires in relation to the size of the input. This comprises:

- **Auxiliary space:** Additional or transient space that the algorithm uses.
 - **Input space:** The amount of space needed to store the input data, albeit this is frequently disregarded in analyses that concentrate on the extra space required.
- For instance:

EXCHANGING THE VALUES OF TWO VARIABLES

Problem

Given two variables a and b, exchange the values assigned to them.

Problem Definition

The problem of changing the values that correspond to two variables involves a fundamental idea that is present in many data manipulation and sorting methods.

Algorithm

Step 1: Start

Step 2: Input: Read two inputs (a and b)

Step 3: Temporary Storage: Use a temporary variable temp.

Step 4: Use swapping technique

- Assign the value of a to temp. (**temp:=a**)
- Assign the value of b to a. (**a:=b**)
- Assign the value of temp to b. (**b:=temp**)

Step 5: The values of a and b are now Exchanged (Display the result).

Step 6: Stop

Programming Technique:

```
temp = a;  
a = b;  
b = temp;
```

COUNTING

"COUNTING" refers to the practice of monitoring numerical values, frequently storing integer values in variables. There are several situations in which counting can be used, including:

1. **Loops:** incrementing a counter variable while iterating a predetermined number of times using loops (such as for, while, or do-while).

```
#include<stdio.h>
```

```
#include<conio.h>
```

```
void main()
```

```
{
```

```
    int count;
```

```
    for (count = 1; count <= 5; count++)
```

```

        {
            printf("%d\n", count);
        }
        getch();
    }

```

Problem Definition

Computer algorithms commonly make use of counting techniques. A count of the number of elements in a set that have a certain property or meet a certain condition or criteria must usually be made. The "examination marks" dilemma is representative of this class of problems.

Algorithm

- Step 1:** Start
- Step 2:** Initialise count:=0
- Step 3:** Enter elements
- Step 4:** Read elements 0 to n-1
Condition (if condition true)
count=count+1
- Step 5:** Display the count
- Step 6:** Stop

Programming Technique:

```

for(i=0;i<n;i++)
{
    If(i>=50)
    {
        count=count+1;
    }
}

```

SUMMATION OF SET OF NUMBERS

Construct an algorithm that accepts a collection of n numbers, adds them up, and then gives back the total. Assume that n is greater than or equal to zero.

The technique we need to take to develop an algorithm to add n numbers in a computer differs from what we would normally do to tackle the problem. Conventionally, we could write the generic equation.

$$\text{sum} = a_1 + a_2 + a_3 + \dots + a_n \quad (3)$$

Algorithm

- Step 1:** Initialize Variables (Create a variable to store the sum (e.g., sum = 0)).
- Step 2:** Input (Decide how many numbers you want to process or input numbers until a sentinel value (like 0) is entered).
- Step 3:** Loop through the Numbers (For each number):
 - Add it to the sum. (**sum=sum + number**)
- Step 4:** Output (Print the total count of numbers and the sum).

Example:

Given Set of Numbers: 5, 10, 15, 20, 25

- **Sum:** 75

Programming Technique:

```
for(i=0;i<n;i++)
{
    sum=sum+number[i];
}
```

FACTORIAL COMPUTATION

Calculating the factorial of a number is a common mathematical operation. The factorial of a non-negative integer n (denoted as $n!$) is the product of all positive integers less than or equal to n .

$$0! = 1$$

$$1! = 1$$

$$2! = 1 * 2 = 2$$

$$3! = 1 * 2 * 3 = 6$$

$$4! = 1 * 2 * 3 * 4 = 24$$

Algorithm

Step 1: Start

Step 2: Input (Read a non-negative integer n).

Step 3: Initialize (Set a variable result to 1 (since the factorial of 0 is 1)).

Step 4: Loop

- For each integer from 1 to n :
- Multiply result by i (**result=result*i**).

Step 4: Output (Print the value of result).

Step 5: Stop

Programming Technique:

```
if(n < 0)
{
    printf("Factorial is not defined for negative numbers:\n");
}
else
{
    // Compute factorial
    for (i = 1; i <= n; i++)
    {
        fact=fact* i; // Multiply current number to the result
    }
}
```

} SINE FUNCTION COMPUTATION

Create an algorithm that uses the infinite series expansion to assess the function $\sin(x)$.

$$\sin(x) = (x/1!) - (x^3/3!) + (x^5/5!) - (x^7/7!) + \dots$$

Analysis

Some of the methods we have seen in previous algorithms are embodied in this problem. When we examine the expression for $\sin(x)$, we see that the factorials and powers follow the

1, 3, 5, 7, ...

are necessary. By starting with 1 and adding 2 one after the other, we may quickly create this unusual sequence. The computation of the generic term $x^i/i!$.

Steps:

- 1. Convert degrees to radians:** Since the sine function works with radians, we first need to convert 38° to radians.

$$X = \text{degree} * (3.142/180)$$

$$X = 38 * (3.142/180)$$

$$= 0.6612 \text{ radian}$$

- 2. Apply the formula**

$$\sin(x) = (x/1!) - (x^3/3!) + (x^5/5!) - (x^7/7!)$$

$$\sin(38^\circ) = (0.6612/1!) - (0.6612^3/3!) + (0.6612^5/5!) - (0.6612^7/7!)$$

$$= (0.6612) - (0.2890/6) + (0.1263/120) - (0.0552/5040)$$

$$= 0.6612 - 0.0481 + 0.001 - 0.00001$$

$$= 0.61409$$

Programming Technique:

```
x=degree*(3.142/180);
```

```
value=x-((x*x*x)/6)+((x*x*x*x*x)/120)-((x*x*x*x*x*x*x)/5040);
```

FIBONACCI SEQUENCE

The Fibonacci series is a sequence of numbers where each number is the sum of the two preceding ones. The sequence usually starts with 0 and 1, though some variations start with 1 and 1. In mathematical terms, the sequence is defined recursively as:

0
1
1
2
3

Algorithm

Step 1: Start

Step 2: Input (Enter how many numbers to display, number>2).

Step 3: Compute new term=1st term+2nd term (**t3=t1+t2**)

Step 4: Loop

- For each integer from 3 to n:
- Display new term (**t3**)
- Exchange the values (**t1=t2 and t2=t3**)
- Add preceding two numbers (**t3=t1+t2**).

Step 5: Stop

Programming Technique:

```
t3=t1+t2;
for(i=3;i<=n;i++)
{
    printf("%d\n",t3);
    t1=t2;
    t2=t3;
    t3=t1+t2;
}
```

REVERSING THE DIGITS OF AN INTEGERS

One typical algorithmic challenge that can be effectively resolved with basic mathematical operations is reversing the digits of an integer.

Compute the Reversing the digits of an Integers:

Digit reversal is a technique that is sometimes used in computing to remove bias from a set of numbers. It is important in some fast information-retrieval algorithms. A specific example clearly defines the relationship of the input to the desired output. For example,

Input: 1234

Output: 4321

Initial number: num = 1234, rev_num = 0.

First iteration:

- digit = 1234 % 10 = 4
- rev_num = 0 * 10 + 4 = 4

- $\text{num} = 1234 / 10 = 123$

Second iteration:

- $\text{digit} = 123 \% 10 = 3$
- $\text{rev_num} = 4 * 10 + 3 = 43$
- $\text{num} = 123 / 10 = 12$

Third iteration:

- $\text{digit} = 12 \% 10 = 2$
- $\text{rev_num} = 43 * 10 + 2 = 432$
- $\text{num} = 12 / 10 = 1$

Fourth iteration:

- $\text{digit} = 1 \% 10 = 1$
- $\text{reversedNum} = 432 * 10 + 1 = 4321$
- $\text{num} = 1 / 10 = 0$

Algorithm:

Step 1: Start

Step 2: Read a number (num)

Step 3: Initialise rev_num=0

Step 4: Loop (Repeat the process until num become 0)

Compute: $\text{digit} = \text{num} \% 10;$
 $\text{rev_num} = \text{rev_num} * 10 + \text{digit};$
 $\text{num} = \text{num} / 10;$

Step 5: Display the result(Stored in rev_num)

Programming Technique:

```
while(num!=0)
{
    rem=num%10;
    rev=rev*10+rem;
    num=num/10;
}
printf("Reversed number=%d",rev);
```

BASE CONVERSION

Computers utilize the number system to represent and manipulate data. Computers utilize binary to process and store data, but depending on the needs of the system—such as

readability, efficiency, or compatibility with human interfaces—other number systems are used at different stages of computation.

Number Systems

A number system is a writing system that uses digits or other symbols to represent numbers consistently. The most widely used number systems in computing are hexadecimal, binary, decimal, and octal. Here's a brief overview:

Binary (Base 2): Uses two digits, 0 and 1, to represent values.

Decimal (Base 10): The standard number system for human counting, using ten digits (0-9).

Octal (Base 8): Uses digits from 0 to 7. Often used in computing as a shorthand for binary.

Hexadecimal (Base 16): Uses digits 0-9 and letters A-F (where A = 10, B = 11, ..., F = 15). This is a compact way to represent binary numbers.

Programming Concept

1. Converting From Binary to Decimal

Initial number: num = 1011, decimal=0, base=1.

First iteration:

- $\text{rem} = 1011 \% 10 = 1$
- $\text{decimal} = 0 + 1 * 1 = 1$
- $\text{num} = 1011 / 10 = 101$
- $\text{base} = 1 * 2 = 2$

Second iteration:

- $\text{rem} = 101 \% 10 = 1$
- $\text{decimal} = 1 + 1 * 2 = 3$
- $\text{num} = 101 / 10 = 10$
- $\text{base} = 2 * 2 = 4$

Third iteration:

- $\text{rem} = 10 \% 10 = 0$
- $\text{decimal} = 3 + 0 * 4 = 3$
- $\text{num} = 10 / 10 = 1$
- $\text{base} = 4 * 2 = 8$

Fourth iteration:

- $\text{rem} = 1 \% 10 = 1$
- $\text{decimal} = 3 + 1 * 8 = 11$
- $\text{num} = 1 / 10 = 0$
- $\text{base} = 8 * 2 = 16$

Algorithm

- Step 1:** Read numbers (Enter only 0's and 1's)
Step 2: Initialise decimal=0, base=1
Step 3: Loop (until number is greater than zero)
 Compute: $\text{rem} = \text{num} \% 10$;
 $\text{decimal} = \text{decimal} + \text{rem} * \text{base}$;
 $\text{num} = \text{num} / 10$;
 $\text{base} = \text{base} * 2$
Step 4: Display the result (decimal)
Step 5: Stop

Programming Technique:

```
while(num>0)
{
    rem= num%10;
    decimal=decimal+rem*base;
    num=num/10;
    base=base*2;
}
```

2. Converting From Octal to Decimal

Initial number: num = 1234, decimal=0, base=1.

First iteration:

- $\text{rem} = 1234 \% 10 = 4$
- $\text{decimal} = 0 + 4 * 1 = 4$
- $\text{num} = 1234 / 10 = 123$
- $\text{base} = 1 * 2 = 2$

Second iteration:

- $\text{rem} = 123 \% 10 = 3$
- $\text{decimal} = 4 + 3 * 2 = 10$
- $\text{num} = 123 / 10 = 12$
- $\text{base} = 2 * 2 = 4$

Third iteration:

- $\text{rem} = 12 \% 10 = 2$
- $\text{decimal} = 10 + 2 * 4 = 18$
- $\text{num} = 12 / 10 = 1$
- $\text{base} = 4 * 2 = 8$

Fourth iteration:

- $\text{rem} = 1 \% 10 = 1$
- $\text{decimal} = 18 + 1 * 8 = 26$
- $\text{num} = 1 / 10 = 0$
- $\text{base} = 8 * 2 = 16$

- $\text{rem} = 1 \% 10 = 1$
- $\text{decimal} = 156 + 1 * 512 = 668$
- $\text{num} = 1 / 10 = 0$
- $\text{base} = 512 * 8 = 4096$

Algorithm

- Step 1:** Read number (Enter only numbers ranges from 0 to 7)
- Step 2:** Initialise $\text{decimal}=0$, $\text{base}=1$
- Step 3:** Loop (until number is greater than zero)
 Compute: $\text{rem}=\text{num}\%10$;
 $\text{decimal} = \text{decimal} + \text{rem}*\text{base}$;
 $\text{num} = \text{num}/10$;
 $\text{base} = \text{base}*8$
- Step 4:** Display the result (decimal)
- Step 5:** Stop

Programming Technique:

```
while(num>0)
{
    rem= num%10;
    decimal=decimal+rem*base;
    num=num/10;
    base=base*8;
}
printf("Decimal number of %d is %d", num, decimal);
```

CHARACTER TO NUMBER CONVERSION

The process of illustrating characters (letters, numbers, symbols, etc.) as numerical values is known as character to number conversion. For activities like encoding, encryption, or mathematical operations, this procedure is crucial in programming and computing. Depending on the system or standard being utilized, there are several ways to accomplish the conversion.

Methods of Character to Number Conversion

1. ASCII (American Standard Code for Information Interchange):

One of the most popular character encoding schemes is ASCII. To represent English characters, numbers, punctuation, and control characters, it assigns a numerical value (ranging from 0 to 127).

Example:

- The letter A is assigned the number 65.
- The letter a is assigned the number 97.
- The digit 1 is assigned the number 49.
- Space is assigned the number 32.

- **Conversion:**

- 'A' -> 65
- 'a' -> 97
- '1' -> 49
- ' ' (space) -> 32

2. FINDING THE SQUARE ROOT OF A NUMBER

To determine the square root of a given number m, first we shall develop an algorithm.

Analysis

When we are first given the task of developing an algorithm to compute square roots, we might not know where to start. In these cases, we need to be very clear about what "the square root of a number" means. For example, we know that 2 squared is equal to 4, 3 squared is equal to 9, 4 squared is equal to 16, and so on. That is,

$$2 \times 2 = 4$$

$$3 \times 3 = 9$$

$$4 \times 4 = 16$$

$$5 \times 5 = 25$$

Algorithm:

Step 1: Start

Step 2: Read a number (n)

Step 3: guess=n/2

Step 4: Loop(Until a==n)

bestGuess=(guess + n /guess)/2

guess=bestGuess

a=guess*guess

Step 5: Stop

Ex: 1

n=25

Iteration 1:

- $\text{guess} = 25 / 2 = 12.5$
- $\text{bestGuess} = (12.5 + 25 / 12.5) / 2 = (12.5 + 2) / 2 = 7.25$

Iteration 2:

- $\text{guess} = 7.25$
- $\text{bestGuess} = (7.25 + 25 / 7.25) / 2 = (7.25 + 3.448) / 2 = 5.349$

Iteration 3:

- $\text{guess} = 5.349$
- $\text{bestGuess} = (5.349 + 25 / 5.349) / 2 = (5.349 + 4.672) / 2 = 5.010$

Iteration 4:

$\text{guess} = 5.010$

$\text{bestGuess} = (5.010 + 25 / 5.010) / 2 = (5.010 + 4.990) / 2 = 5.000$

Programming Technique:

```

if (num < 0)
{
    printf("You entered a negative number\n");
}
else
{
    while (a!=num)
    {
        bestGuess = (guess + num / guess) / 2;
        guess = bestGuess;
        a=guess*guess;
    }
}
printf("Square root of %f is %f",num,bestGuess);
}

```

THE SMALLEST DIVISOR OF AN INTEGER

Create an algorithm that, given an integer n , will determine its smallest exact divisor that is not 1.

Analysis

This problem appears to be fairly trivial when taken at face value. The set of numbers 2, 3, 4,..., n can be divided into n by dividing each one in turn. Since we must have determined the smallest exact divisor of n , our algorithm can end as soon as we come across a number

in the set that divides into n exactly. All of this is very simple. But the question still stands: is it possible to create an algorithm that is more effective?

Algorithm:

Step 1: Start

Step 2: Take input n ($n > 1$).

Step 3: Check Divisibility by 2:

- If n is divisible by 2 (i.e., $n \bmod 2 = 0$), then 2 is the smallest divisor of n .

Step 4: Check Divisibility by Odd Numbers:

- If n is not divisible by 2, check divisibility starting from 3, and increment by 2 (to only check odd numbers).
- For each odd number d , check if $n \bmod d = 0$. If it does, then d is the smallest divisor of n .
- You can stop the search as soon as d exceeds $\sqrt{n}$, because if n does not have a divisor smaller than or equal to $\sqrt{n}$, then n itself must be prime, and n is the smallest divisor of itself.

Step 5: Display answer.

Step 6: Stop

Programming Technique:

```
if (n % 2 == 0)
{
    printf("The smallest divisor of %d is: 2\n", n);
}
else
{
    for (i = 3; i <= sqrt(n); i += 2)
    {
        if (n % i == 0)
        {
            divisor = i;
            break;
        }
    }
    if (divisor != 0)
    {
        printf("The smallest divisor of %d is: %d\n", n, divisor);
    }
    else
```

```

{
    printf("The smallest divisor of %d is: %d\n", n, n);
}
}
}

```

THE GREATEST COMMON DIVISOR OF TWO INTEGERS

The largest positive integer that divides two integers without leaving a remainder is the Greatest Common Divisor (GCD), sometimes referred to as the Greatest Common Factor (GCF) or Highest Common Factor (HCF).

Analysis:

When we first encounter this issue, we notice that , it differs slightly from other issues we have most likely faced. The relationship between the divisors of two numbers is the challenging part of the problem. Therefore, we might start by decomposing the problem and independently determining all of the divisors of the two integers, n and m. We quickly discover that we need to choose the largest element shared by both lists of divisors once we have these two lists. The greatest common divisor of the two integers, n and m, must be this element.

$$\text{Dividend} = \text{Divisor} * \text{Quotient} + \text{Remainder}$$

For example:

GCD of 132 and 320

The diagram illustrates the Euclidean algorithm for finding the GCD of 132 and 320. It shows a series of division steps with remainders and corresponding equations:

- $320 = 132(2) + 56$
- $132 = 56(2) + 20$
- $56 = 20(2) + 16$
- $20 = 16(1) + 4$
- $16 = 4(4) + 0$

The final non-zero remainder, 4, is identified as the H.C.F. (Highest Common Factor).

Dividend	Divisor	Quotient	Remainder	Equation
320	132	2	56	$320=132*2+56$
132	56	2	20	$132=56*2+20$
56	20	2	16	$56=20*2+16$

20	16	1	4	$20=16*1+4$
16	4	4	0	$16=4*4+0$

Algorithm:

Step 1: Start

Step 2: Read two positive integers a and b (where $a \geq b$).

Step 3: If $b=0$, then returns a (This is the base case).

Step 4: Loop (until $b \neq 0$)

 Compute the remainder $r=a \bmod b$.

 Replace a with b, and b with r.

Step 7: Display the answer (a).

Step 8: Stop

Programming Technique:

```

while (b != 0)
{
    r = a % b;
    a = b;
    b = r;
}
printf("GCD is %d\n", a);
}

```

GENERATING PRIME NUMBERS

A **prime number** is a natural number greater than 1 that cannot be formed by multiplying two smaller natural numbers. In simpler terms, it is a number that has exactly two distinct positive divisors: 1 and itself. This means that a prime number cannot be divided evenly by any other number except for 1 and itself.

Analysis:

To check if a number n is prime, follow these steps:

1. **If n is less than 2**, it is not prime (e.g., 0, 1, and negative numbers are not prime).
2. **If $n=2$** , it is prime (2 is the smallest prime number).

3. **For numbers greater than 2**, check divisibility by numbers from 2 up to the square root of n.
 - If any number between 2 and the square root of n divides n evenly (i.e., the remainder is 0), then n is **not prime**.
 - If no such number exists, then n is **prime**.

Example 1: Is 29 Prime?

To determine if 29 is prime:

- Check divisibility by all numbers up to $29 \approx 5.3929 \approx 5.39$. So, check divisibility by 2, 3, and 5.
 - 29 is not divisible by 2 (since it's odd).
 - 29 is not divisible by 3 (sum of digits $2 + 9 = 11$, not divisible by 3).
 - 29 is not divisible by 5 (it doesn't end in 0 or 5).

Since 29 is not divisible by any of these numbers, it is **prime**.

Algorithms:

Step 1: Start

Step 2: Read a number

Step 3: Loop(num=2 to n)

Initialise count=0

Loop(i=2 to num/2)

check (num mod i == 0)

count increment

check (count==0 and num!=1)

print num

Step 4: Stop

Programming Technique:

```
for (num = 2; num <= n; num++)
{
    count=0;
    for (i = 2; i <= num/2; i++)
    {
        if (num % i == 0)
        {
            count++;
            break;
        }
    }
    if (count==0 && num!=1)
    {
        printf("%d ", num);
    }
}
```

}

COMPUTING THE PRIME FACTORS OF AN INTEGER

You must continually divide an integer by its smallest prime divisor until you are unable to divide any more in order to determine its prime factors. By beginning with the smallest prime (2) and working through the remaining prime numbers, one can determine the smallest prime divisor.

Steps:

1. **Start with the smallest prime (2):** Check if the number is divisible by 2, and if it is, divide the number by 2 repeatedly until it is no longer divisible by 2.
2. **Move to the next odd numbers:** Once you've divided out all factors of 2, continue checking divisibility with odd numbers starting from 3.
3. **Stop when the number becomes 1:** Once you've divided the number completely, the remaining factor (if greater than 1) will be prime.
4. **Examples:**

2	24
2	12
2	6
3	3
	1

Program

```

for(i=2; i<=num; i++)
{
    while(num%i==0)
    {
        printf("%d\t", i);
        num=num/i;
    }
}

```

::

2	324
2	162
3	81
3	27
3	9
3	3
	1

GENERATION OF PSEUDO-RANDOM NUMBERS

A number that looks to be random but is actually produced using a deterministic method is called a pseudo-random number. Although the numbers produced by this technique are not completely random, they are sufficient for the majority of uses, including games, simulations, and cryptography (although cryptographic random number generators are recommended for high-security applications).

The rand() function in the standard library (stdlib.h) of C can be used to produce pseudo-random numbers.

rand() :

The pseudo-random numbers are produced by the rand() function. It uses a beginning value known as the seed to generate a series of numbers. If the seed is left unset, the software will automatically create the same set of integers each time it runs. You must use the srand() function to set the seed, usually with a changing value, such as the current time, in order to guarantee distinct sequences of random integers.

1. **rand() :** Returns a pseudo-random integer between 0 and `RAND_MAX` (a constant defined in `<stdlib.h>`).
2. **srand() :** Sets the seed for `rand()`. By setting a different seed, you get a different sequence of random numbers.

RAISING A NUMBER TO A LARGE POWER

It can be computationally difficult to raise a number to a large power directly, particularly when b is huge. For example, computing a^b , where a is the base and b is the exponent, is an example of this. Exponentiation by Squaring is a popular method for quickly computing big powers since it minimizes the amount of multiplications needed.

This process is recursive or iterative and reduces the problem size by half at each step.

COMPUTING n^{th} FIBONACCI NUMBER

The Fibonacci sequence is a series of numbers where each number is the sum of the two preceding ones, typically starting with 0 and 1. The nth Fibonacci number can be defined as:

$$F(0)=0,$$

$$F(1)=1$$

$$F(n)=F(n-1)+F(n-2) \quad \text{for } n>2$$

Approaches

1. **Recursive Approach**
2. **Iterative Approach**

1. Recursive Approach :

The naive recursive approach follows the definition of the Fibonacci sequence directly. However, it is not efficient for large n because it recalculates values multiple times, resulting in an exponential time complexity $O(2^n)$.

2. Iterative Approach

An iterative approach is much more efficient with a time complexity of $O(n)$ and a space complexity of $O(1)$.

ARRAY TECHNIQUES

Arrays:

An array is a collection of elements, all of the same datatype, stored in contiguous memory locations. You define an array by specifying its data type and the number of elements it will hold.

Syntax:

Data type arrayName [arraySize];

Example:

```
int num[5]; //It holds 5 elements of integer type
```

ARRAY ORDER REVERSAL

To make an array appear in reverse order, rearrange its elements.

Analysis

It seems like a really simple problem to reverse the order of an array of numbers. Although this is basically true, the algorithm's implementation requires major consideration and care.

By carefully looking at an array's elements both before and after they have been reversed, we can begin designing this technique. For instance,

Before reversal

1	2	3	4	5	6	7
---	---	---	---	---	---	---

After reversal

7	6	5	4	3	2	1
---	---	---	---	---	---	---

Algorithm:

Step 1: Start

Step 2: Read n and array elements:

Step3: Initialize two pointers:

- $i = 0$ (beginning of the array)
- $j = n - 1$ (end of the array)

Step 4: Loop (While $i < j$)

- $temp = a[i]$
- $a[i] = a[j]$
- $a[j] = temp$
- increment i and decrement j

Step 5: Continue until the i pointer is no longer less than j (i.e., $i \geq j$).

Step 6: The array is now reversed in place.

Step 7: Stop

Problem Technique:

```
i = 0;
j = n - 1;
while (i < j)
{
    temp = a[i];
    a[i] = a[j];
    a[j] = temp;
    i++;
    j--;
}
```

ARRAY COUNTING

Counting instances of certain values or items within an array is the usual use of the array counting notion. An array's items can be counted in a variety of methods, but the most popular ones are as follows:

1. Counting Elements with Specific Conditions:

- Determine the number of elements in the array that meet a specific requirement (e.g., elements greater than a specified value).

Algorithm: Count Frequency of Specific Element in an Array

Step 1: Start

Step 2: Initialize count=0

Step 3: Read n and array elements

Step 4: Read specific element need to count (Element).

Step 5: Loop

Check (Element==a[i])

 Count=count+1 (if true)

Step 6: Display the count (i.e Number of time Element is present)

Step 7: Stop

FINDING THE MAXIMUM NUMBER IN A SET

In a set of n numbers, find the maximum number.

Analysis:

We must have a clear understanding of what a maximum is before we can start working on the technique for determining it. After giving it some thought, we may determine that the maximum is the number that exceeds or equals every other number in the set. The possibility that the maximum is not unique is taken into account by this definition. It also suggests that only sets consisting of one or more items are deemed to have a maximum.

Algorithm:

Step 1: Start

Step 2: Read n and array elements.

Step 3: Set or initialise max=a[0](i.e first element)

Step 4: Loop

 If(a[i]>max)

 Then max=a[i];

Step 5: Display the ans (max).

Step 6: Stop

Program Technique:

```
max=a[0];
for (i = 0; i < size; i++)
{
    if (arr[i] > max)
    {
        max = arr[i];
    }
}
printf("The maximum element in the array is: %d\n", max);
```

REMOVAL OF DUPLICATES FROM AN ORDERED ARRAY

An ordered array should be compressed once all duplicates have been eliminated.

Analysis

Let's start this design by concentrating on a particular example so that we can clearly see what is needed.

Before duplicate removal

1	2	3	4	5	6	7	8	9	10	11	12	13
2	2	8	15	23	23	23	23	26	29	30	32	32

We may create the specified array below after taking a quick look at the array:

After duplicate removal

1	2	3	4	5	6	7	8
2	8	15	23	26	29	30	32

Ex

Array a: {1, 1, 2, 3, 3, 4, 5, 5, 6}

Initial values:

arr = [1, 1, 2, 3, 3, 4, 5, 5, 6]

newIndex = 1, i = 1

First Iteration (i = 1):

- Compare a[1] (1) with a[0] (1). They are equal, so skip.
- Move to next iteration.

Second Iteration (i = 2):

- Compare a[2] (2) with a[1] (1). They are not equal, so:
- Place a[2] at a[newIndex] (i.e., a[1] = 2).
- Increment newIndex to 2.

Third Iteration (i = 3):

- Compare a[3] (3) with a[2] (2). They are not equal, so:
- Place a[3] at a[newIndex] (i.e., a[2] = 3).
- Increment newIndex to 3.

Fourth Iteration (i = 4):

- Compare a[4] (3) with a[3] (3). They are equal, so skip.

Fifth Iteration (i = 5):

- Compare a[5] (4) with a[4] (3). They are not equal, so:
- Place a[5] at a[newIndex] (i.e., a[3] = 4).
- Increment newIndex to 4.

Sixth Iteration (i = 6):

- Compare a[6] (5) with a[5] (4). They are not equal, so:
- Place a[6] at a[newIndex] (i.e., a[4] = 5).

- Increment newIndex to 5.

Seventh Iteration (i = 7):

- Compare a[7] (5) with a[6] (5). They are equal, so skip.

Eighth Iteration (i = 8):

- Compare a[8] (6) with a[7] (5). They are not equal, so:
- Place a[8] at a[newIndex] (i.e., a[5] = 6).
- Increment newIndex to 6.

Algorithm:

Step 1: Start

Step 2: Read n and Array elements

Step 3: Initialise newIndex = 1

Step 4: loop (Read elements index value1 to n-1)

 Check(a[i] != a[i - 1])

 a[newIndex] = a[i];

 increment newIndex by 1;

Step 5: Print the new elements (removed duplicate elements)

Step 6: Stop

Program Technique:

```
if (n == 0 || n == 1)
{
    printf("Array after removing duplicates: \n");
    for (i = 0; i < n; i++)
    {
        printf("%d ", a[i]);
    }
    printf("\n");
    return 0;
}
newIndex = 1;
for (i = 1; i < n; i++)
{
    if (a[i] != a[i - 1])
    {
        a[newIndex] = a[i];
        newIndex++;
    }
}
printf("Array after removing duplicates:\n");
for (i = 0; i < newIndex; i++)
{
```

```
        printf("%d ", a[i]);  
    }
```

PARTITION OF ARRAY

Rearranging an array's components in accordance with a certain condition or set of criteria is known as partitioning. Although the idea of partitioning is utilized in a variety of applications, including data segmentation, dynamic programming, and searching, it is particularly significant in sorting algorithms like Quick Sort and Merge Sort.

Types of Array Partitioning:

Balanced Partitioning (divide into sub arrays of roughly equal size)

Partition for Quick Sort (based on a pivot element)

Partition into Two Groups (e.g., based on some condition, like less than or greater than a value)

Partitioning with Odd and Even Numbers.

BALANCED PARTITIONING (Divide into sub arrays of roughly equal size)

Balanced partitioning refers to dividing an array into two sub arrays that are as equal in size as possible. This is commonly used in various algorithms, such as **Merge Sort**, where the array is recursively split into smaller sub arrays. In balanced partitioning, the goal is to ensure that the two sub arrays are roughly equal in size, with at most a difference of one element.

Ex:

$a[] = \{10, 20, 30, 40, 50, 60, 70\}$

Calculate the midpoint:

$mid = 7 / 2 = 3$ (integer division).

First Half:

The first half will contain elements from index 0 to 2: {10, 20, 30}.

Second Half:

The second half will contain elements from index 3 to 6: {40, 50, 60, 70}.

Output:

First half: 10 20 30

Second half: 40 50 60 70

Algorithm:

PROF. Mithun

Global College of Computer Applications, Hubli

Step 1: Start

Step 2: Read n and array elements

Step 3: initialise mid=0,low=0 and high=n-1

Step 4: Compute mid(mid=low+high)

Step 5: Loop

Print elements from index value 0 to mid-1(First part)

Print elements from index value mid to high (Second part)

Step 6: Stop

Programming Technique:

```
low=0;
high=n-1;
mid=(low+high)/2;
printf("First half: ");
for (i = 0; i < mid; i++)
{
    printf("%d ", a[i]);
}
printf("Second half: ");
for (i = mid; i < n; i++)
{
    printf("%d ", a[i]);
}
```

FINDING THE k^{th} SMALLEST ELEMENT

The k-th smallest element in an array is simply the element that would appear in the k-th position if the array were sorted in ascending order. For example, in a sorted array, the 1st smallest element is the smallest number, the 2nd smallest element is the second smallest number, and so on.

Algorithm:

Step 1: Start

Step 2: Read n and array elements.

Step 3: Read kth Position

Step 4: Arrange the elements in ascending order

n = length of array

for i from 0 to n-1:

```

swapped = false
for j from 0 to n-i-2:
    if arr[j] > arr[j + 1]:
        swap arr[j] and arr[j + 1]
        swapped = true
    if not swapped:
        break # Array is already sorted

```

Step 5: Display the kth position element

Step 6: Stop

Programming Technique:

```

for (i = 0; i < n - 1; i++)
{
    for (j = 0; j < n - i - 1; j++)
    {
        if (a[j] > a[j + 1])
        {
            temp = a [j];
            a [j] = a [j + 1];
            a [j + 1] = temp;
        }
    }
}
printf("The %d smallest element is %d\n", k, a[k - 1]);

```

MERGING, SORTING, SEARCHING

INTRODUCTION

Sorting, finding, and merging are related tasks that are essential to many computer programs. According to some estimates, sorting alone takes up almost 30% of all computer time.

Sorting and merging give us a way to arrange data so that certain information may be retrieved more easily. In order to minimize the amount of work required to either find a specific item or prove that it is absent from a data set, searching techniques are made to capitalize on the arrangement of information.

THE TWO-WAY MERGE

Sorting algorithms like Merge Sort are the main applications for the two-way merge principle. It describes how two sorted sequences (lists or arrays) can be combined into one sorted sequence. A crucial step in combining sorted subarrays back into a single, bigger sorted array is the "two-way" merge operation, which denotes that two sorted sets are being merged.

The two-way merge step works as follows:

- You have two sorted subarrays (or lists).
- You need to combine these two subarrays into a single sorted array, maintaining the order of elements.

Example:

Two sorted arrays:

- Array A: [1, 3, 5]
- Array B: [2, 4, 6]

The goal of a two-way merge is to combine these two arrays into a single sorted array: [1, 2, 3, 4, 5, 6].

1. Initialization:

- You start with two pointers (or indices), i and j , initialized to the beginning of the two arrays ($i = 0$ for Array A and $j = 0$ for Array B).
- You also need a result array (`merged[]`) where the final sorted list will be stored.

2. Comparison:

- Compare the elements pointed to by i and j ($A[i]$ and $B[j]$).
- Insert the smaller element into the `merged[]` array and move the pointer of the array from which the smaller element was taken.

3. Continue Merging:

- Repeat the comparison process until you've gone through all elements of both arrays.
- If one array is exhausted (i.e., all elements from that array have been added to `merged[]`), simply append the remaining elements from the other array to `merged[]` (since the remaining elements in that array are already sorted).

SORTING BY SELECTION

The selection sort algorithm repeatedly finds the smallest element from the unsorted part and inserts it at the beginning of the array with the ascending order in view.

Example:

5	9	6	1	2	8
---	---	---	---	---	---

1. First iteration (i = 0):

- The initial unsorted array is: [5,9,6,1,2,8]
- We find the smallest element in the subarray [5, 9, 6, 1, 2], which is 1.
- Swap 1 with 5.
- Array after first iteration: [1, 9, 6, 5, 2,8]

5	9	6	1	2	8
---	---	---	---	---	---

1	9	6	5	2	8
---	---	---	---	---	---

2. Second iteration (i = 1):

- Now consider the unsorted subarray [9,6,5,2,8].
- The smallest element is 2.
- Swap 2 with 9.
- Array after second iteration: [1,2,6,5,9,8]

1	9	6	5	2	8
---	---	---	---	---	---

1	2	6	5	9	8
---	---	---	---	---	---

3. Third iteration (i = 2):

- Now consider the unsorted subarray [6,5,9,8].
- The smallest element is 5.
- Swap 5 with 6.
- Array after third iteration: [1,2,5,6,9,8]

1	2	6	5	9	8
---	---	---	---	---	---

1	2	5	6	9	8
---	---	---	---	---	---

4. Third iteration (i = 3):

- Now consider the unsorted subarray [6,9,8].
- The smallest element is 6, which is already in the correct position
- No swap needed.
- Array after third iteration: [1,2,5,6,9,8]

1	2	5	6	9	8
---	---	---	---	---	---

1	2	5	6	9	8
---	---	---	---	---	---

5. Fourth iteration (i = 4):

- Now consider the unsorted subarray [9,8].
- The smallest element is 8.
- Swap 8 with 9.
- Final sorted array: [1,2,5,6,8,9]

1	2	5	6	9	8
---	---	---	---	---	---

1	2	5	6	9	8
---	---	---	---	---	---

Algorithm:

Step 1: Start

Step 2: Read size of array and elements

Step 3: Loop

1. Start from the first element (i = 0).

2. Repeat the following steps for each element from $i = 0$ to $i = n-2$ (since the last element will be placed in its correct position automatically):
 - a. Set the minimum index (min) to i .
 - b. Find smallest element in the unsorted portion of the array (from index i to $n-1$):
 - Iterate over the array starting from $j = i+1$ to $j = n-1$.
 - If $a[j]$ is smaller than $a[\text{min}]$, update min to j .
 - c. **Swap the smallest element** found ($a[\text{min}]$) with the element at index i ($a[i]$), if min is not equal to i . This places the smallest element in the correct position in the sorted portion of the array.

Step 4: Display sorted array

Step 5: Stop

SORTING BY EXCHANGE

It is the simplest sorting algorithm that works repeatedly by swapping the adjacent elements if they are in wrong order. This algorithm is not suitable for large data sets as its average and worst-case time complexities is quite high.

Stepping over the list frequently, comparing neighbouring items, and switching them if they are out of order is how it operates. The list is sorted when no more swaps are required, which is achieved by repeating the operation.

Example 1:

5	3	8	4	2
---	---	---	---	---

Pass 1:

- Compare 5 and 3: $5 > 3$, so swap them. New array: [3, 5, 8, 4, 2]
- Compare 5 and 8: $5 < 8$, no swap needed. Array stays: [3, 5, 8, 4, 2]
- Compare 8 and 4: $8 > 4$, so swap them. New array: [3, 5, 4, 8, 2]
- Compare 8 and 2: $8 > 2$, so swap them. New array: [3, 5, 4, 2, 8]
- After Pass 1, the largest element (8) is in its correct position. New array: [3, 5, 4, 2, 8]

Pass 2:

- Compare 3 and 5: $3 < 5$, no swap needed. Array stays: [3, 5, 4, 2, 8]
 - Compare 5 and 4: $5 > 4$, so swap them. New array: [3, 4, 5, 2, 8]
 - Compare 5 and 2: $5 > 2$, so swap them. New array: [3, 4, 2, 5, 8]
 - After Pass 2, the second largest element (5) is in its correct position. New array: [3, 4, 2, 5, 8]
-

Pass 3:

- Compare 3 and 4: $3 < 4$, no swap needed. Array stays: [3, 4, 2, 5, 8]
 - Compare 4 and 2: $4 > 2$, so swap them. New array: [3, 2, 4, 5, 8]
 - After Pass 3, the third largest element (4) is in its correct position. New array: [3, 2, 4, 5, 8]
-

Pass 4:

- Compare 3 and 2: $3 > 2$, so swap them. New array: [2, 3, 4, 5, 8]
- After Pass 4, the second smallest element (3) is in its correct position. New array: [2, 3, 4, 5, 8]

Now, the list is fully sorted as [2, 3, 4, 5, 8].

SORTING BY INSERTION

The array is split into two halves in order for the insertion sort to function:

- The **sorted part**, which initially contains just the first element (since a single element is trivially sorted).
- The **unsorted part**, which contains the rest of the elements.
- One by one, the algorithm selects items from the unsorted section and places them in the appropriate location within the sorted section.

Step 1

- Assume the first element of the array is already sorted.
- The unsorted part consists of the remaining elements.

Step 2

1. Pick the element (this is called key) to insert into the sorted part of the array.
2. Compare the key with elements in the sorted part, starting from the rightmost element of the sorted part.
3. Shift elements: As long as the elements in the sorted part are greater than the key, shift them one position to the right to make space for the key.
4. Insert the key into its correct position in the sorted part.

Example

Consider the array:

12	11	13	5	6
----	----	----	---	---

- **Initial Array:** {12, 11, 13, 5, 6}
 - The first element 12 is already considered sorted.
 - The unsorted part is {11, 13, 5, 6}.

Iteration 1:

- **Key** = 11 (second element).
- We compare 11 with 12 (the only element in the sorted part).
- Since $11 < 12$, we shift 12 to the right to make space for 11.
- We insert 11 into the correct position: {11, 12, 13, 5, 6}.
- Sorted part after first iteration: {11, 12}.

12	11	13	5	6
----	----	----	---	---

11	12	13	5	6
----	----	----	---	---

Iteration 2:

- **Key** = 13 (third element).

- We compare 13 with 12 (last element in the sorted part).
- Since $13 > 12$, no shifting is needed, and 13 is already in the correct position.
- Sorted part after second iteration: {11, 12, 13}.

11	12	13	5	6
----	----	----	---	---

11	12	13	5	6
----	----	----	---	---

Iteration 3:

- **Key** = 5 (fourth element).
- We compare 5 with 13, 12, and 11, and find that 5 is smaller than all of them.
- We shift 13, 12, and 11 to the right to make space for 5.
- Insert 5 into the correct position: {5, 11, 12, 13, 6}.
- Sorted part after third iteration: {5, 11, 12, 13}.

11	12	13	5	6
----	----	----	---	---

5	11	12	13	6
---	----	----	----	---

Iteration 4:

- **Key** = 6 (fifth element).
- We compare 6 with 13, 12, 11, and 5.
- We shift 13, 12, and 11 to the right because they are greater than 6, but 5 is already smaller than 6.
- Insert 6 into the correct position: {5, 6, 11, 12, 13}.
- Sorted part after fourth iteration: {5, 6, 11, 12, 13}.

5	11	12	13	6
---	----	----	----	---

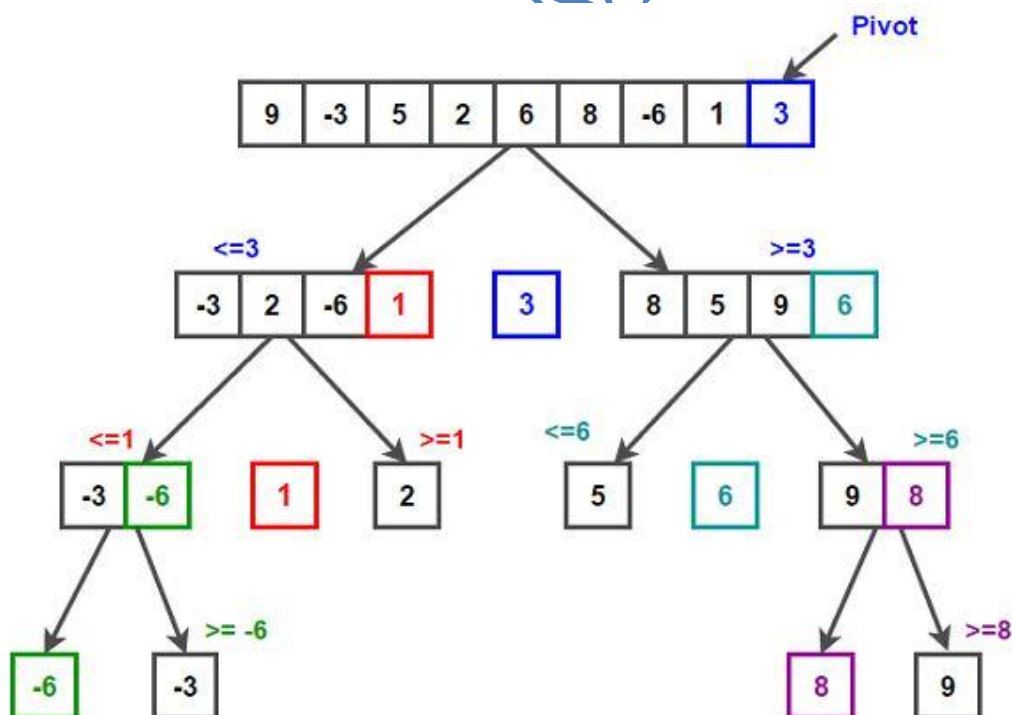
5	6	11	12	13
---	---	----	----	----

SORTING BY PARTITIONING

Sorting algorithms that employ the divide-and-conquer strategy: Here, the array is recursively split into smaller components, then combined or processed in a way that guarantees the entire array is sorted—are referred to as sorting by partitioning. QuickSort is the most well-known partitioning method.

QuickSort (Sorting by Partitioning)

QuickSort is a highly efficient **divide-and-conquer** algorithm that works by selecting a **pivot** element from the array and then **partitioning** the array around the pivot such that:



Algorithm:

1. **Choose a Pivot:**
 - Select an element from the array to be the pivot. There are different ways to choose the pivot (e.g., first element, last element, middle element, or random element).
2. **Partition the Array:**
 - Rearrange the array such that:
 - All elements less than the pivot go to the left of the pivot.
 - All elements greater than the pivot go to the right of the pivot.
 - After partitioning, the pivot is now in its correct position in the sorted array.
3. **Recursively Sort the Subarrays:**
 - Apply the same process to the left and right subarrays (elements smaller than the pivot and elements larger than the pivot).
 - Repeat until each subarray has one or zero elements (which are naturally sorted).

Quick Sort - Simple Algorithm:

1. **Pick a pivot** from the array.
2. **Partition** the array into two sub-arrays based on the pivot.
3. **Recursively apply** the same steps to the left and right subarrays.

LINEAR SEARCH

One of the most basic search algorithms is linear search, referred as sequential search. By sequentially inspecting each element in an unordered list (or array) until the target element (element to be searched) is located or the entire list has been traversed, it is used to locate a specific element.

Step 1: Start at the beginning of the array: Begin the search from the first element of array.

Step 2: Compare the current element with the target: Check if the current element matches

the target value you are looking for.

Step 3: If a match is found: If the current element matches the target, return the index (position) of that element in the array, and the search is complete.

Step 4: If no match is found: Move to the next element in the array.

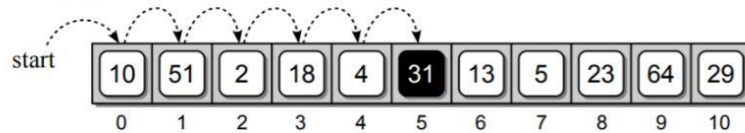
Step 5: Repeat steps 2-4: Continue checking each element in sequence until either a match

is found or all elements are checked.

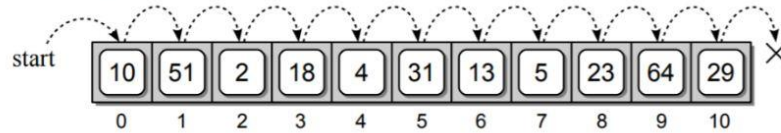
Step 6: End of the array: If the entire array is traversed without finding the target, return a

value indicating that the target is not in the array (typically -1 or None).

(a) Searching for 31



(b) Searching for 8



Algorithm:

Step 1: Start

Step 2: Initialise pos =0

Step 3: Read size of array and array elements

Step 4: Read key element (searching element)

Step 5: Loop (Read a[0] to a[n-1])

 Check (a[i]==key)

 pos++

 Break

Step 6: Check (pos !=0) Element is found

 Otherwise element is not found

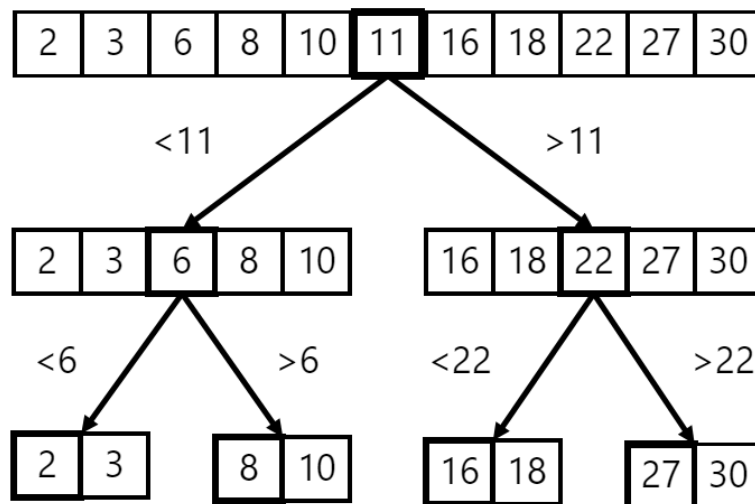
Step 7: Stop

BINARY SEARCH

An effective method for locating a desired value within a sorted array or list is binary search. It divides the search interval in half repeatedly. The search proceeds in the left half if the target value is less than the value in the middle of the interval; if not, it proceeds in the right half. Because half of the remaining components are removed at each stage, binary search is significantly faster than linear search, particularly for big datasets.

1. Begin with two pointers, usually low for the array's beginning and high for its end.
2. Middle Element: Use the formula $\text{mid} = (\text{low} + \text{high})/2$ to determine the middle index of the current search interval.
3. Comparison:
 - Return the middle element's index if the middle element equals the key value.

- Move the high pointer to mid-1 (pay attention to the left half) if the middle element is larger than the key element.
 - Move the low pointer to mid + 1 (pay attention to the right half) if the middle element is smaller than the key element.
4. Repeat: Continue the process until the low pointer exceeds the high pointer, meaning the target value is not in the array.



2	3	6	8	10	11	16	18	22	27	30
---	---	---	---	----	----	----	----	----	----	----

Key Element: 2

No. of Elements: 11

1. Initialize Pointers:

- Start with two pointers: low = 0 (the first element) and high = 10 [n-1] the last element).

2. First Step (Calculate Middle Element):

- Calculate the middle index using the formula: $\text{mid} = (\text{low} + \text{high}) / 2$
- $\text{mid} = (0 + 10) / 2 = 5$
- The element at index 5 is 11.

Compare key element with Middle Element:

- We check if 11 (the middle element) is equal to the key element 2.
- Since 11 is greater than 2, we know the key element must be in the left half of the array. Therefore, we adjust the high pointer to $\text{mid} - 1$, which is $5 - 1 = 4$.

3. Second Step (New Search Interval):

- Now, the new search interval is between $\text{low} = 0$ and $\text{high} = 4$. The sub-array is [2, 3, 6, 8, 10].
- Calculate the new middle index: $\text{mid} = (0+4)/2=2$
- The element at index 2 is 6.

Compare Key element with Middle Element:

- We check if 6 (the middle element) is equal to the key element 2.
- Since 6 is greater than 2, we know the key element must be in the left half of the array. Therefore, we adjust the high pointer to $\text{mid} - 1$, which is $2 - 1 = 1$.

4. Third Step (New Search Interval):

- Now, the new search interval is between $\text{low} = 0$ and $\text{high} = 1$. The sub-array is [2, 3].
- Calculate the new middle index: $\text{mid} = (0+1)/2=0$
- The element at index 0 is 2.

5. Key element is Found:

Prof. Mithun